ExceptionLab Activity

Answers:

Activity A

```java
String name = null;

name.length();
```

- **Exception:** `NumberFormatException`
- **Classification:** Unchecked exception.
- **Explanation:** The `parseInt()` method attempts to convert the string `"ABC"` into an integer, but ABC is not a valid number.
- **Handling strategy:** Use a `try-catch` block to catch `NumberFormatException` and display a friendly message informing the user that the input must be numeric.

Answer:

```java
String name = null;

name.length();
```

- **Exception:** `NumberFormatException`
- **Classification:** Unchecked exception.
- **Explanation:** The `parseInt()` method attempts to convert the string `"ABC"` into an integer, but ABC is not a valid number.
- **Handling strategy:** Use a `try-catch` block to catch `NumberFormatException` and display a friendly message informing the user that the input must be numeric.

```java
new java.io.FileReader("missig.txt")
```

Answer:

- **Exception:** `FileNotFoundException`
- **Classification:** Checked exception.
- **Explanation:** The exception occurs when the specified file cannot be found or opened.
- **Handling strategy:** Use a `try-catch` block to catch `FileNotFoundException` and display a helpful message to the user.

```java
Explain in 1–2 sentences why OutOfMemoryError should not normally be caught simply to continue execution.
```

Answer:

`OutOfMemoryError` occurs when the Java Virtual Machine cannot allocate the memory required by the program. Catching it and continuing execution may cause further failures because the program may not have enough memory to operate safely.

ACTIVITY D:

Answer:

`IOException` and its subclass `FileNotFoundException` are checked exceptions. The `throws IOException` declaration appears in the `importMarks()` method because file-reading operations may produce an `IOException`.

The declaration allows the calling method, `main()`, to handle the exception.

`throw` is used to explicitly throw an exception when an invalid condition occurs, while `throws` declares exceptions that a method may pass to its caller.

Example of `throw`:

Friendly messages help users understand the problem without requiring technical knowledge. The logger records technical details, such as exception types and causes, to help developers identify and fix problems during debugging.

An empty `catch` block silently ignores an exception, making it difficult to identify errors and potentially allowing the program to continue incorrectly.

Catching `Exception` before `FileNotFoundException` makes the specific catch block unreachable because `FileNotFoundException` is a subclass of `Exception`.

```
package org.example;

import java.io.*;
import java.util.*;
import java.util.logging.Logger;
import java.util.logging.Level;

public class MarksImporter {

   // Logger stores technical details for debugging
   private static final Logger logger =
           Logger.getLogger(MarksImporter.class.getName());

   // Custom exception for duplicate student IDs
   static class DuplicateStudentException extends RuntimeException {

       public DuplicateStudentException(String message) {
           super(message);
       }
   }

   // Imports marks from the text file
   static void importMarks(String filename) throws IOException {

       Set<String> seen = new HashSet<>();
```

```
// Try-with-resources automatically closes the reader,
// even if an exception occurs.
try (BufferedReader br =
             new BufferedReader(new FileReader(filename))) {

    String line;
    int lineNumber = 0;

    while ((line = br.readLine()) != null) {

        lineNumber++;

        try {

            // TODO 1: Check that there are exactly 2 fields
            String[] parts = line.split(",", -1);

            if (parts.length != 2) {
                throw new IllegalArgumentException(
                        "Record must contain student ID and mark");
            }

            String studentId = parts[0].trim();
            String markText = parts[1].trim();

            if (studentId.isEmpty()) {
                throw new IllegalArgumentException(
                        "Student ID cannot be empty");
            }

            // TODO 2: Parse and validate the mark
            int mark = Integer.parseInt(markText);

            if (mark < 0 || mark > 100) {
                throw new IllegalArgumentException(
                        "Mark must be between 0 and 100");
            }

            // TODO 3: Detect duplicate student IDs
            if (seen.contains(studentId)) {
                throw new DuplicateStudentException(
                        "Duplicate student ID: " + studentId);
            }

            seen.add(studentId);

            // TODO 4: Display accepted record
            System.out.println(
```

```
                        "Accepted: " + studentId + " = " + mark);

            } catch (NumberFormatException e) {

                System.out.println(
                        "Invalid numeric mark on line " + lineNumber
                                + ": " + line);

                logger.log(Level.WARNING,
                        "NumberFormatException on line " + lineNumber,
                        e);

            } catch (DuplicateStudentException e) {

                System.out.println(
                        "Duplicate record skipped: " + line);

                logger.log(Level.WARNING,
                        "Duplicate student record on line "
                                + lineNumber, e);

            } catch (IllegalArgumentException e) {

                System.out.println(
                        "Invalid record on line " + lineNumber
                                + ": " + e.getMessage());

                logger.log(Level.WARNING,
                        "Invalid data on line " + lineNumber, e);
            }
        }
    }
}

// Main method: program execution starts here
public static void main(String[] args) {

    try {

        // T5: Testing a missing file
        importMarks("absent.txt");

    } catch (FileNotFoundException e) {

        System.out.println(
                "Marks file not found; check the file name");

        logger.log(Level.SEVERE,
                "FileNotFoundException: " + e.getMessage(), e);
```

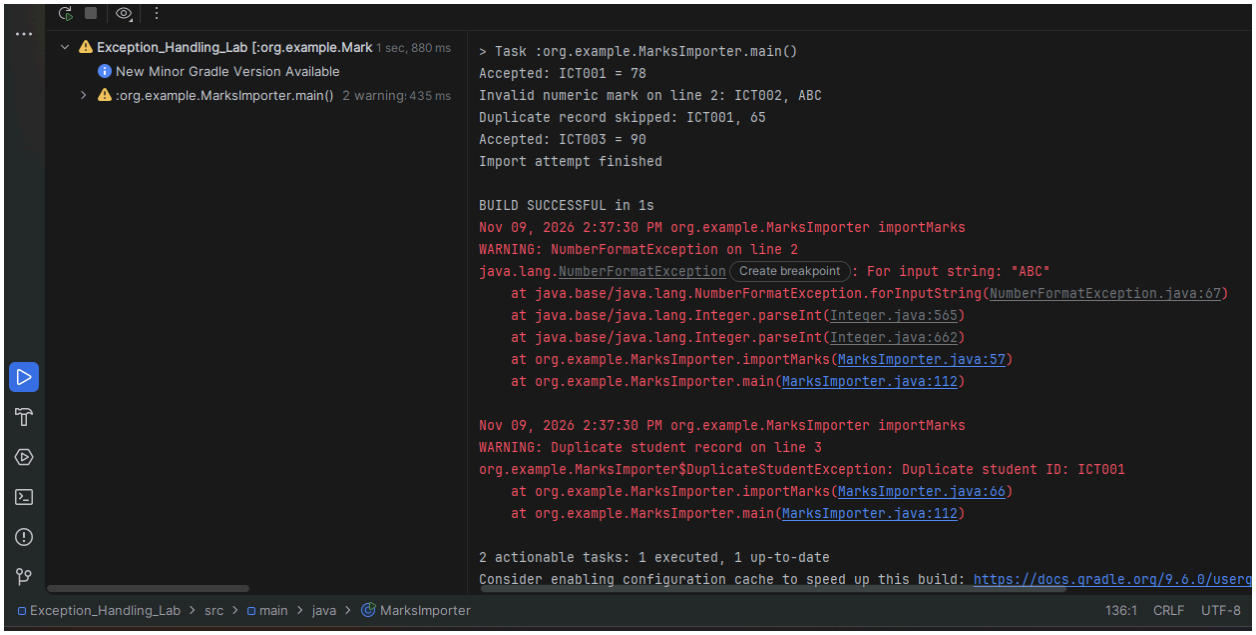
```
        } catch (IOException e) {

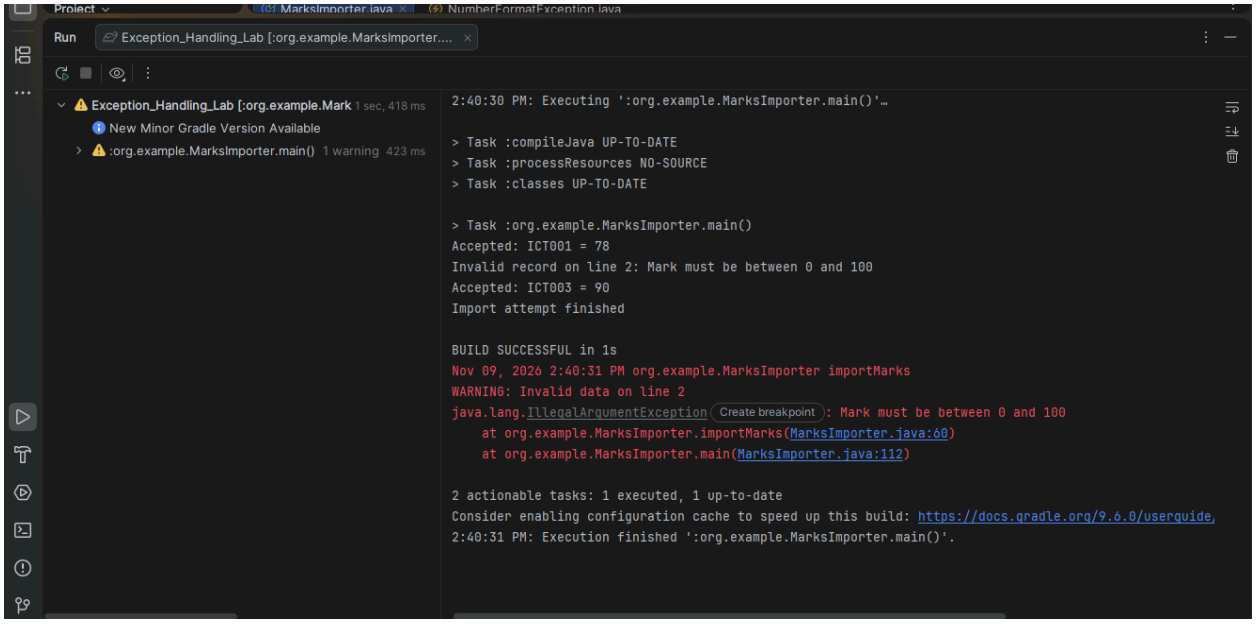
            System.out.println(
                    "An error occurred while reading the marks file");

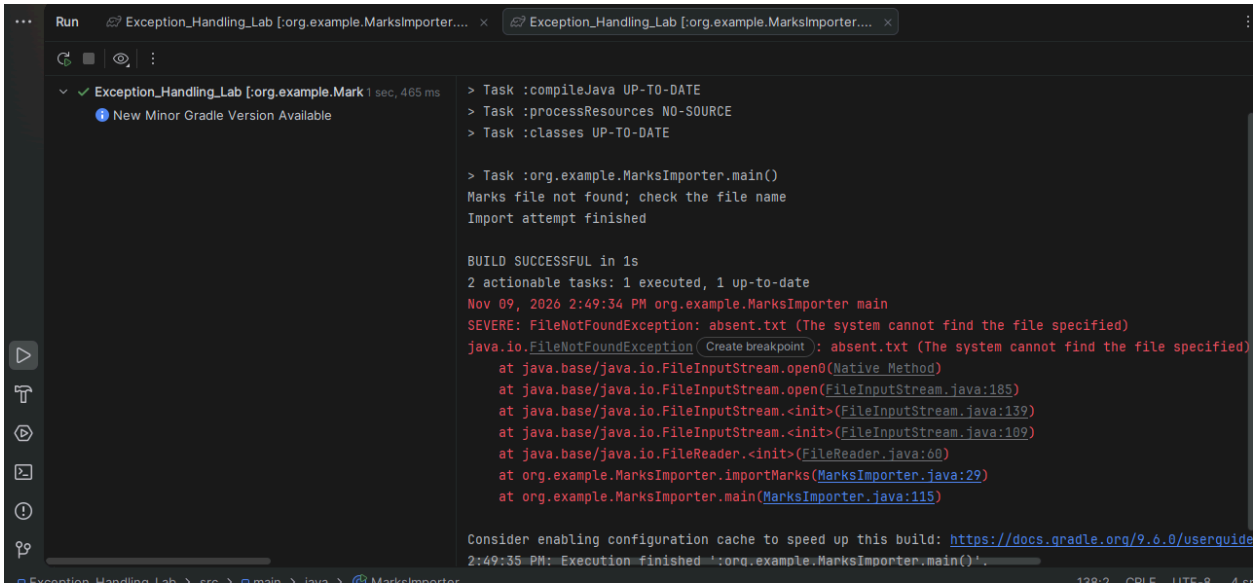
            logger.log(Level.SEVERE,
                    "IOException while importing marks", e);

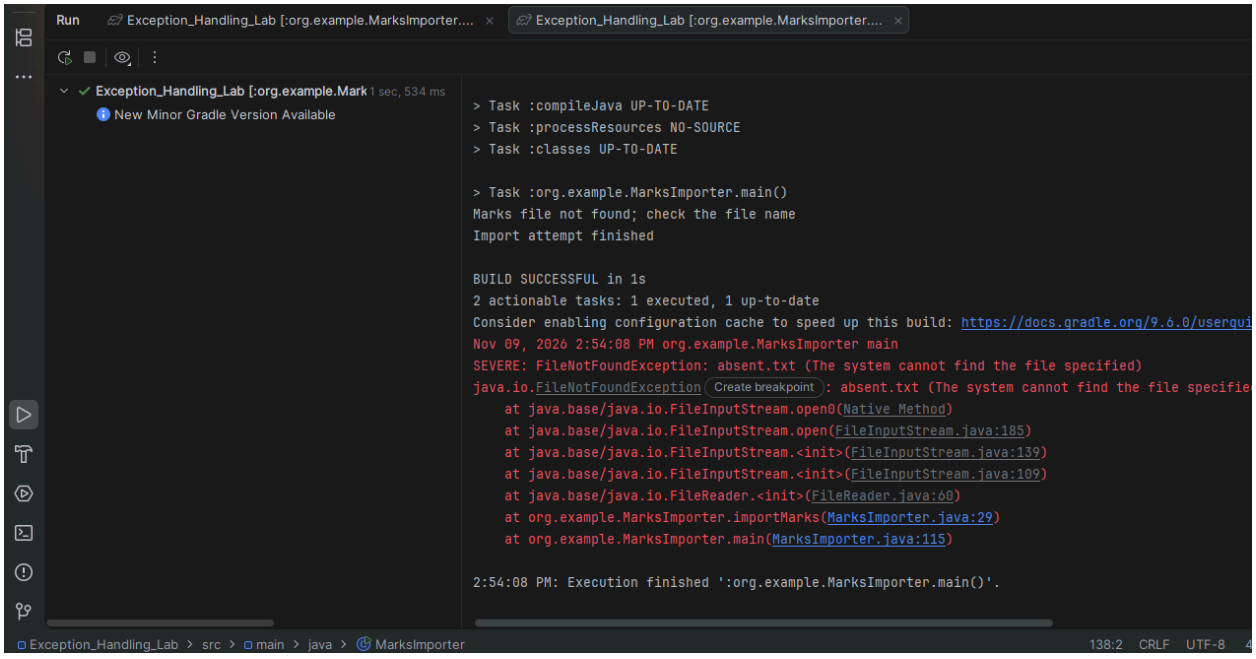
        } finally {

            System.out.println("Import attempt finished");
        }
    }
}
```


Exception_Handling_Lab [:org.example.Mark 1 sec, 880 ms
New Minor Gradle Version Available
:org.example.MarksImporter.main() 2 warning: 435 ms
> Task :org.example.MarksImporter.main()
Accepted: ICT001 = 78
Invalid numeric mark on line 2: ICT002, ABC
Duplicate record skipped: ICT001, 65
Accepted: ICT003 = 90
Import attempt finished
BUILD SUCCESSFUL in 1s
Nov 09, 2026 2:37:30 PM org.example.MarksImporter importMarks
WARNING: NumberFormatException on line 2
java.lang.NumberFormatException Create breakpoint : For input string: "ABC"
at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
at java.base/java.lang.Integer.parseInt(Integer.java:565)
at java.base/java.lang.Integer.parseInt(Integer.java:662)
at org.example.MarksImporter.importMarks(MarksImporter.java:57)
at org.example.MarksImporter.main(MarksImporter.java:112)
Nov 09, 2026 2:37:30 PM org.example.MarksImporter importMarks
WARNING: Duplicate student record on line 3
org.example.MarksImporter$DuplicateStudentException: Duplicate student ID: ICT001
at org.example.MarksImporter.importMarks(MarksImporter.java:66)
at org.example.MarksImporter.main(MarksImporter.java:112)
2 actionable tasks: 1 executed, 1 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.6.0/userg
Exception_Handling_Lab > src > main > java > MarksImporter
136:1 CRLF UTF-8


Run
Exception_Handling_Lab [:org.example.MarksImporter....
Exception_Handling_Lab [:org.example.Mark 1 sec, 418 ms
New Minor Gradle Version Available
:org.example.MarksImporter.main() 1 warning 423 ms
2:40:30 PM: Executing ':org.example.MarksImporter.main()'…
> Task :compileJava UP-TO-DATE
> Task :processResources NO-SOURCE
> Task :classes UP-TO-DATE
> Task :org.example.MarksImporter.main()
Accepted: ICT001 = 78
Invalid record on line 2: Mark must be between 0 and 100
Accepted: ICT003 = 90
Import attempt finished
BUILD SUCCESSFUL in 1s
Nov 09, 2026 2:40:31 PM org.example.MarksImporter importMarks
WARNING: Invalid data on line 2
java.lang.IllegalArgumentException Create breakpoint : Mark must be between 0 and 100
at org.example.MarksImporter.importMarks(MarksImporter.java:60)
at org.example.MarksImporter.main(MarksImporter.java:112)
2 actionable tasks: 1 executed, 1 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.6.0/userguide,
2:40:31 PM: Execution finished ':org.example.MarksImporter.main()'.


Run
Exception_Handling_Lab [:org.example.MarksImporter....
Exception_Handling_Lab [:org.example.MarksImporter....
Exception_Handling_Lab [:org.example.Mark 1 sec, 465 ms
New Minor Gradle Version Available
> Task :compileJava UP-TO-DATE
> Task :processResources NO-SOURCE
> Task :classes UP-TO-DATE
> Task :org.example.MarksImporter.main()
Marks file not found; check the file name
Import attempt finished
BUILD SUCCESSFUL in 1s
2 actionable tasks: 1 executed, 1 up-to-date
Nov 09, 2026 2:49:34 PM org.example.MarksImporter main
SEVERE: FileNotFoundException: absent.txt (The system cannot find the file specified)
java.io.FileNotFoundException Create breakpoint : absent.txt (The system cannot find the file specified)
at java.base/java.io.FileInputStream.open0(Native Method)
at java.base/java.io.FileInputStream.open(FileInputStream.java:185)
at java.base/java.io.FileInputStream.<init>(FileInputStream.java:139)
at java.base/java.io.FileInputStream.<init>(FileInputStream.java:109)
at java.base/java.io.FileReader.<init>(FileReader.java:60)
at org.example.MarksImporter.importMarks(MarksImporter.java:29)
at org.example.MarksImporter.main(MarksImporter.java:115)
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.6.0/userguide
2:49:35 PM: Execution finished ':org.example.MarksImporter.main()'.


Run
Exception_Handling_Lab [:org.example.MarksImporter....
Exception_Handling_Lab [:org.example.MarksImporter....
Exception_Handling_Lab [:org.example.Mark 1 sec, 534 ms
New Minor Gradle Version Available
> Task :compileJava UP-TO-DATE
> Task :processResources NO-SOURCE
> Task :classes UP-TO-DATE
> Task :org.example.MarksImporter.main()
Marks file not found; check the file name
Import attempt finished
BUILD SUCCESSFUL in 1s
2 actionable tasks: 1 executed, 1 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.6.0/usergui
Nov 09, 2026 2:54:08 PM org.example.MarksImporter main
SEVERE: FileNotFoundException: absent.txt (The system cannot find the file specified)
java.io.FileNotFoundException Create breakpoint : absent.txt (The system cannot find the file specifie
at java.base/java.io.FileInputStream.open0(Native Method)
at java.base/java.io.FileInputStream.open(FileInputStream.java:185)
at java.base/java.io.FileInputStream.<init>(FileInputStream.java:139)
at java.base/java.io.FileInputStream.<init>(FileInputStream.java:109)
at java.base/java.io.FileReader.<init>(FileReader.java:60)
at org.example.MarksImporter.importMarks(MarksImporter.java:29)
at org.example.MarksImporter.main(MarksImporter.java:115)
2:54:08 PM: Execution finished ':org.example.MarksImporter.main()'.
Exception_Handling_Lab > src > main > java > MarksImporter
138:2 CRLF UTF-8